Lecture 9

INSTRUCTION SET ARCHITECTURE

(ISA)

Introduction

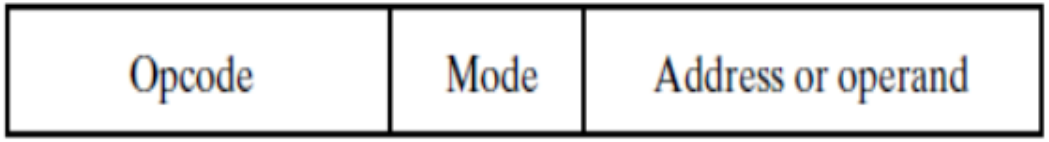
- In this chapter, we will study material dealing with instruction set architecture for general-purpose computers.
- We will examine the operations that the instructions perform and focus particularly on how the operands are obtained and where the results are stored.
- We will contrast two distinct classes of architectures: reduced instruction set computers (RISCs) and complex instruction set computers (CISCs).

Cont.,

- The binary language in which instructions are defined and stored in memory is referred to as *machine language*.
- A symbolic language that replaces binary opcodes and addresses with symbolic names is referred to as *assembly language*.
- A computer usually has a variety of instructions and multiple instruction formats.

Instruction Format

- Instruction format is a binary format which specifies a computer instruction.
- The format of an instruction is depicted in a rectangular box symbolizing the bits of the binary instruction.
- The bits are divided into groups called *fields*.
- The following are typical fields found in instruction formats:



Cont.,

1. An *opcode field*, which specifies the operation to be performed.

2. An *address field*, which provides either a memory address or an address that selects a processor register.

3. A *mode field*, which specifies the way the address field is to be interpreted.

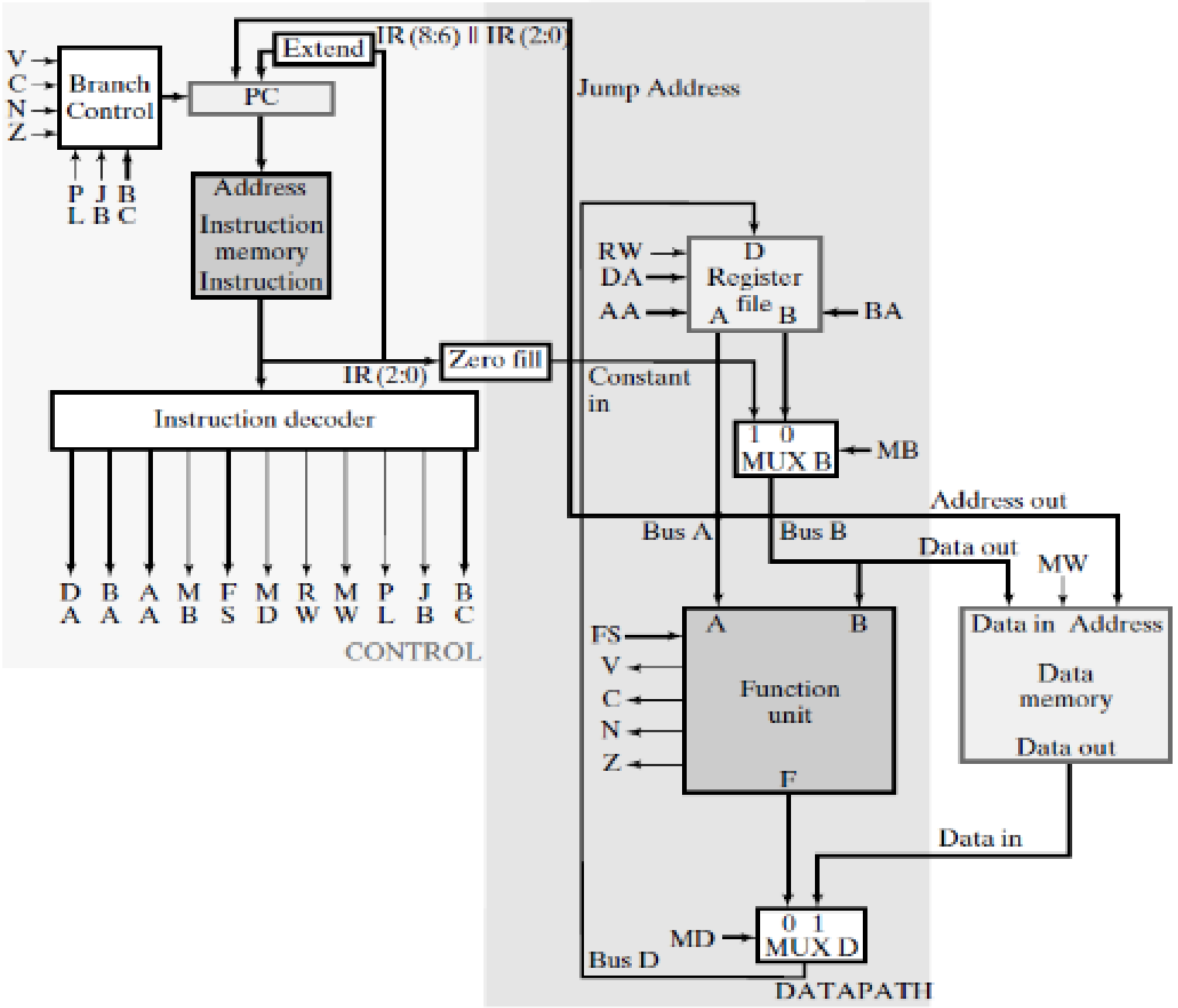
Basic Computer Operation Cycle

The computer's control unit is designed to execute each instruction of a program in the following sequence of steps:

- Fetch the instruction from memory (Program counter).
- Decode the instruction (Instruction Decoder).
- Locate the operands used by the instruction.
- Fetch operands from memory (if necessary).
- Execute the operation in processor registers.
- Store the results in the proper place.
- Go back to step 1 to fetch the next instruction.

Note

- A register in the computer called the *program counter* (*PC*) keeps track of the instructions in the program stored in memory.
- The *PC* holds the address of the instruction to be executed next and is incremented each time a word is read from the program in memory.
- The decoding done in Step 2 determines the operation to be performed and the addressing mode or modes of the instruction.
- The operands in Step 3 are located from the addressing modes and the address fields of the instruction.
- The computer executes the instruction, storing the result, and returns to Step 1 to fetch the next instruction in sequence.


V
C
N
Z
Branch Control
PL
JB
BC
PC
Extend
IR (8:6) || IR (2:0)
Jump Address
Address
Instruction memory
Instruction
IR (2:0)
Zero fill
Constant in
Instruction decoder
RW
DA
AA
D
Register file
A
B
BA
1 0
MUX B
MB
Address out
Bus A
Bus B
Data out
MW
DA BA AA MB FS MD RW MW PL JB BC
CONTROL
FS
V
C
N
Z
A
B
Function unit
F
Data in
Address
Data memory
Data out
Data in
MD
0 1
MUX D
Bus D
DATAPATH

Instructions Classification according to operand addressing:

1. Three-Address Instructions

- Consider an instruction such as ADD, which specifies the addition of two operands to produce a result.
- Suppose that the result of the addition is treated as just another operand.
- Then the ADD instruction has three operands.

ADD T1, A, B , $M[T1] \leftarrow M[A] + M[B]$

2- Two-Address Instructions

- For two-address instructions, each address field can again specify either a possible register or a memory address.
- The first operand address listed in the symbolic instruction also serves as the implied address to which the result of the operation is transferred.
- The program is as follows:

MOVE T1, A	$M[T1] \leftarrow M[A]$
ADD T1, B	$M[T1] \leftarrow M[T1] + M[B]$
MOVE X, C	$M[X] \leftarrow M[C]$
ADD X, D	$M[X] \leftarrow M[X] + M[D]$
MUL X, T1	$M[X] \leftarrow M[X] \times M[T1]$

3- One-Address Instructions

LD	A	$ACC \leftarrow M[A]$
ADD	B	$ACC \leftarrow ACC + M[B]$
ST	X	$M[X] \leftarrow ACC$
LD	C	$ACC \leftarrow M[C]$
ADD	D	$ACC \leftarrow ACC + M[D]$
MUL	X	$ACC \leftarrow ACC \times M[X]$
ST	X	$M[X] \leftarrow ACC$

4- Zero-Address Instructions

- To perform an ADD instruction with zero addresses, all three addresses in the instruction must be implied.
- A conventional way of achieving this goal is to use a ***stack***, which is a mechanism or structure that stores information such that the item stored last is the first retrieved.

ADDRESSING MODES

- The operation field of an instruction specifies the operation to be performed.
- This operation must be executed on data stored in computer registers or memory words.
- How the operands are selected during program execution is dependent on the addressing mode of the instruction.
- An addressing mode is how we tell the processor where to find the data that it needs to use with each instruction.

1- Immediate Mode

- In the immediate mode, the operand is specified in the instruction itself. It has an operand field rather than an address field.
- Immediate-mode instructions are useful, for example, for initializing registers to a constant value.
- Ex:

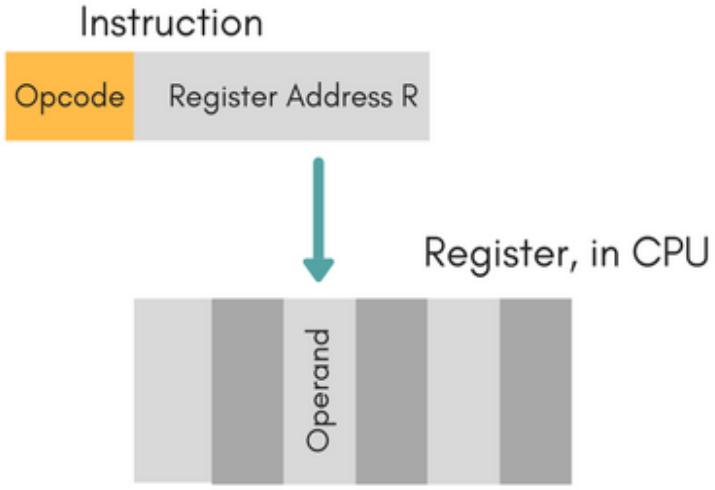
  MOV AX, 0005H

Examples

Assembly Language	*Size*	*Operation*
MOV BL,44	8 bits	Copies 44 decimal (2CH) into BL
MOV AX,44H	16 bits	Copies 0044H into AX
MOV SI,0	16 bits	Copies 0000H into SI
MOV CH,100	8 bits	Copies 100 decimal (64H) into CH
MOV AL,'A'	8 bits	Copies ASCII A into AL

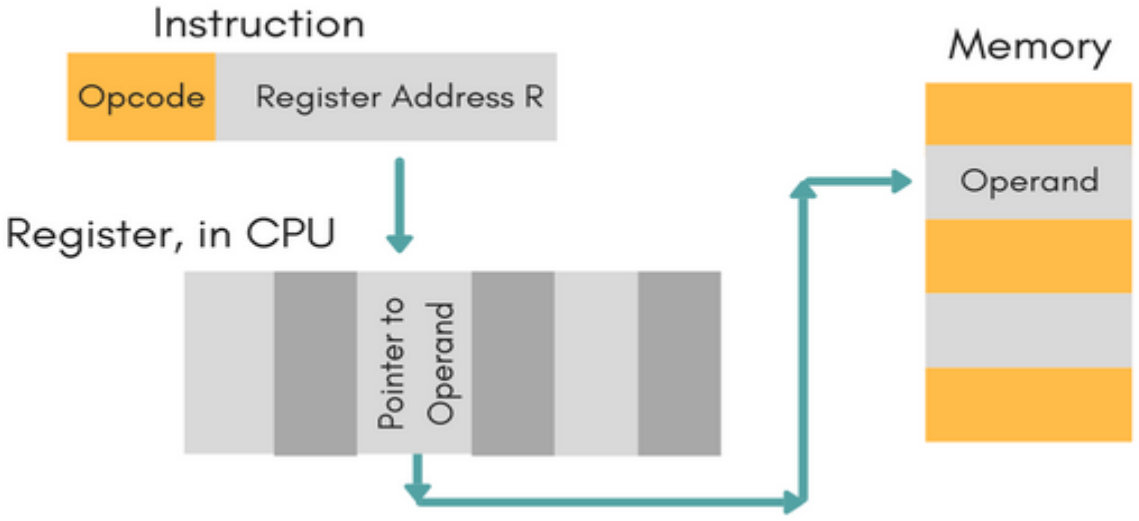
2- Register Direct Addressing

- When the address field specifies a processor register, the instruction is said to be in the register mode. In this mode, the operands are in registers that reside within the processor of the computer.
- Ex: MOV AX, BX



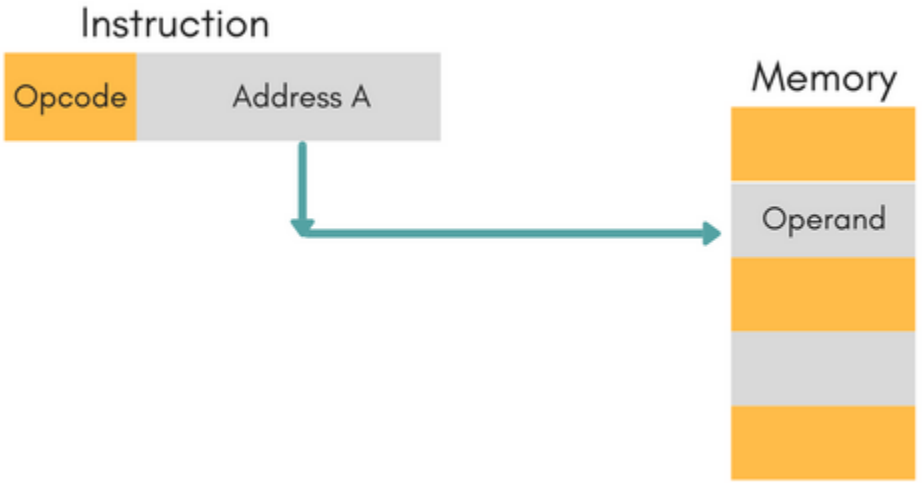
3- Register-indirect mode

- In the register-indirect mode, the instruction specifies a register in the processor whose content gives the address of the operand in memory.
- Ex: MOV AX, [BX]



4- Direct Addressing Mode

- In the direct addressing mode, the address field of the instruction gives the address of the operand in memory in a data-transfer or data-manipulation instruction.
- Ex: ADD R1, 000F



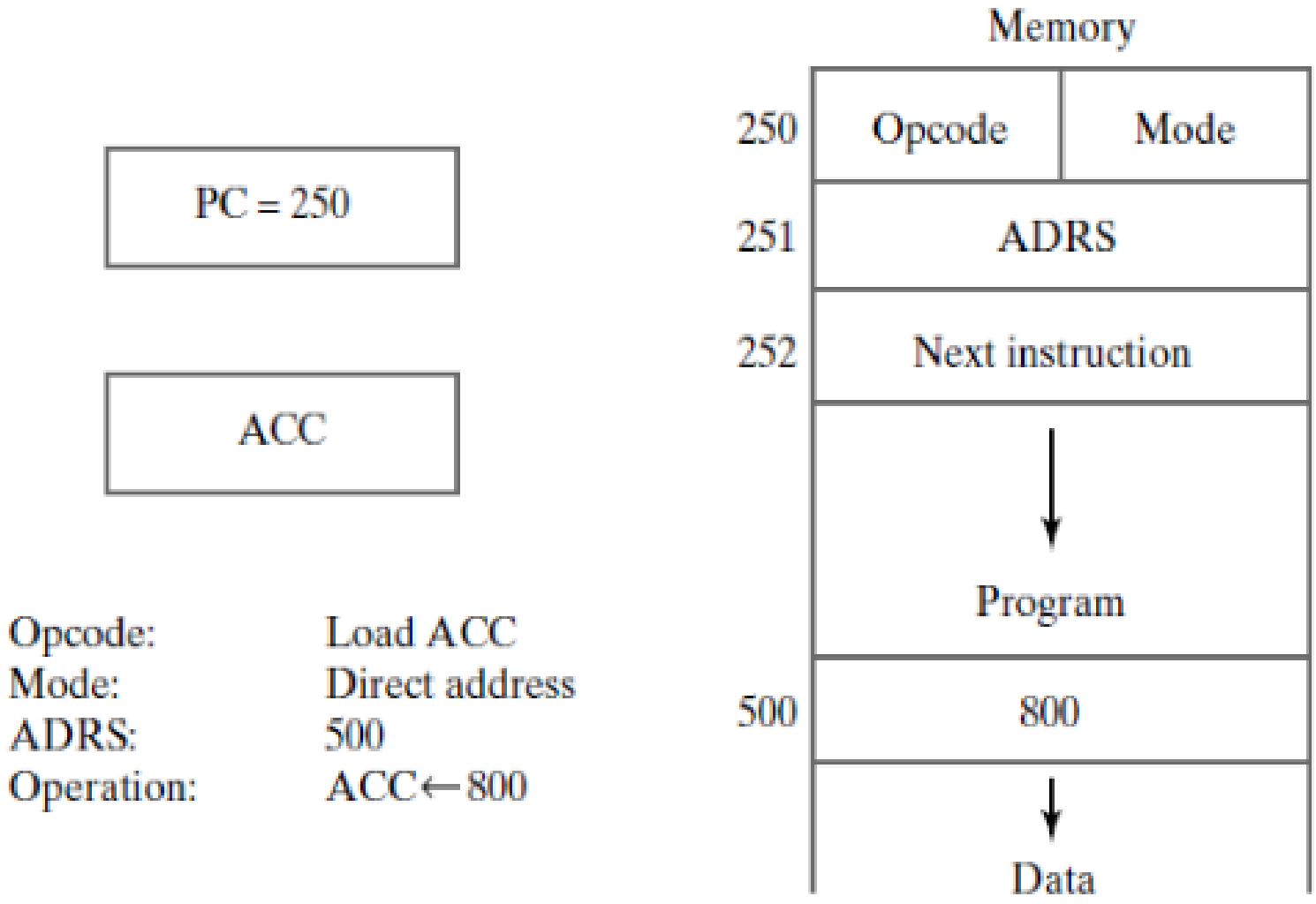
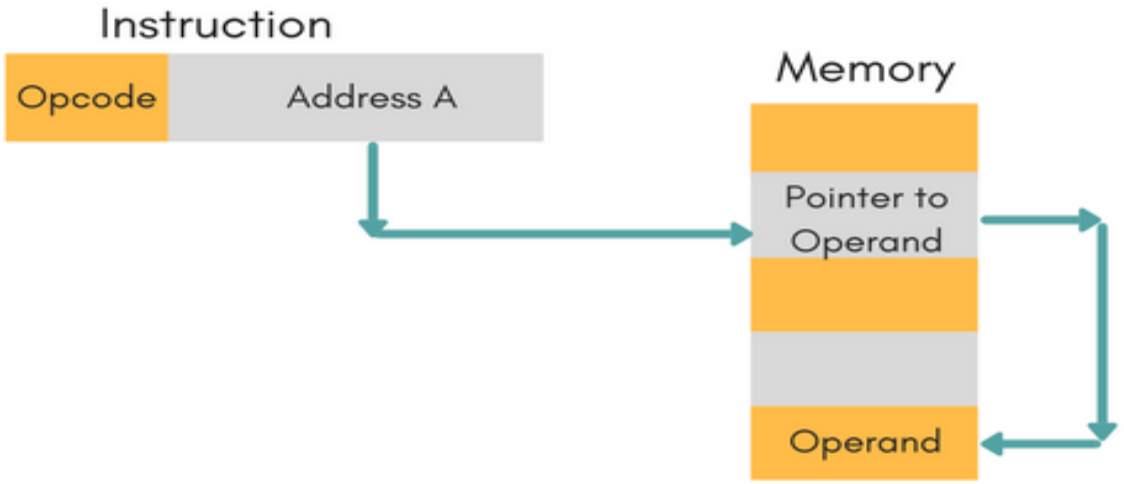


FIGURE 4
Example Demonstrating Direct Addressing for a Data-Transfer Instruction

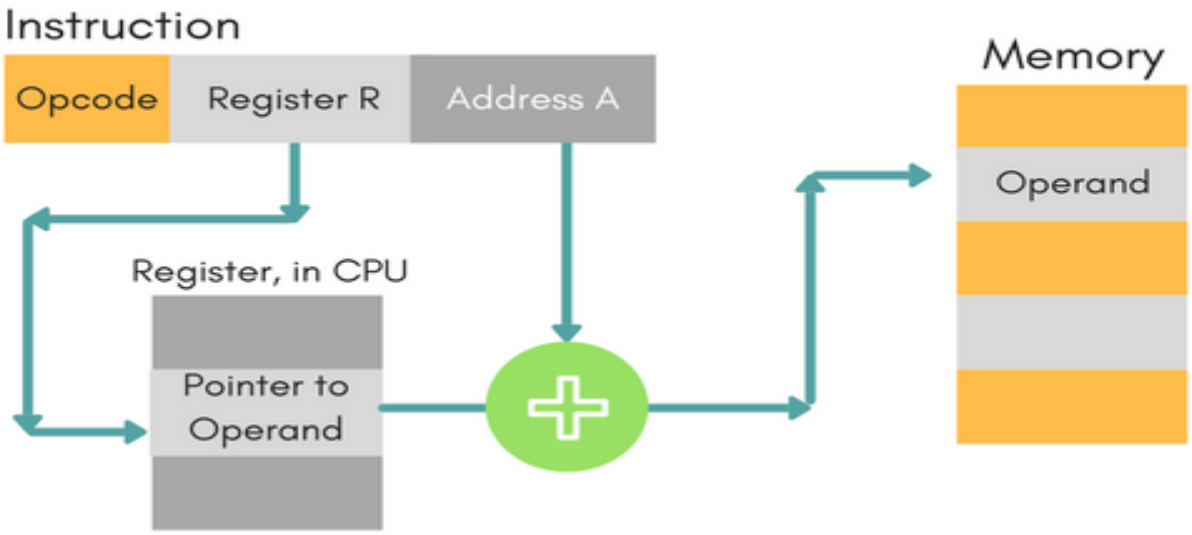
5- Indirect Addressing Mode

- In the indirect addressing mode, the address field of the instruction gives the address at which the effective address is stored in memory.



6- Indexed Addressing Mode

- The content of an index register is added to the address part of the instruction to obtain the effective address.
- The index register may be a special CPU register or simply a register in a register file.



7-Relative Addressing Mode

- Relative addressing is often used in branching.
- Effective address = Address part of the instruction + Contents of *PC*

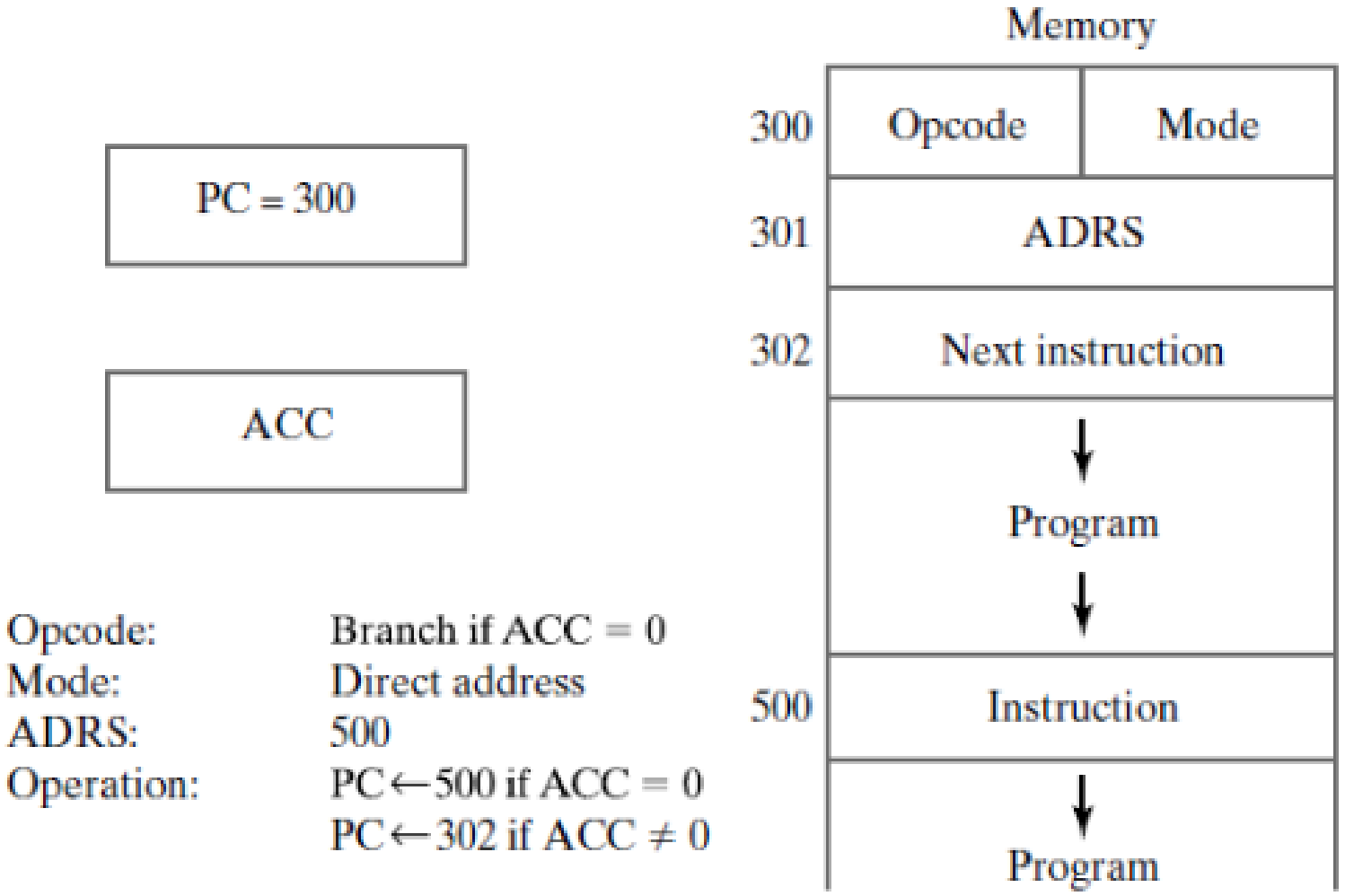


FIGURE 5
Example Demonstrating Direct Addressing in a Branch Instruction

TABLE 1
Symbolic Convention for Addressing Modes

Addressing Mode	Symbolic Convention	Register Transfer
Direct	LDA ADRS	$ACC \leftarrow M[ADRS]$
Immediate	LDA #NBR	$ACC \leftarrow NBR$
Indirect	LDA [ADRS]	$ACC \leftarrow M[M[ADRS]]$
Relative	LDA $ADRS	$ACC \leftarrow M[ADRS + PC]$
Index	LDA ADRS (R1)	$ACC \leftarrow M[ADRS + R1]$
Register	LDA R1	$ACC \leftarrow R1$
Register-indirect	LDA (R1)	$ACC \leftarrow M[R1]$

Note

- Table 1 lists the value of the effective address and the operand loaded into the *ACC* for the seven addressing modes.
- The table also shows the operation with a register transfer statement and a symbolic convention for each addressing mode.
- LDA is the symbol for the load-to-accumulator opcode.
- In the direct mode, we use the symbol ADRS for the address part of the instruction.
- The # symbol precedes the operand NBR (is a number or operand) in the immediate mode.
- The symbol ADRS enclosed in square brackets symbolizes an indirect address, which some compilers or assemblers designate with the symbol @.

Cont.,

- The symbol $ before the address makes the effective address relative to the *PC.*
- An index-mode instruction is recognized by the symbol of a register placed in parentheses after the address symbol.
- The register mode is indicated by giving the name of the processor register following LDA.
- In the register- indirect mode, the name of the register that holds the effective address is enclosed in parentheses.

Complex Instruction Set Computer (CISC)

A purely *CISC architecture* has the following properties:

1. Memory access is directly available to most types of instructions.

2. Addressing modes are substantial.

3. Instruction formats are of different lengths.

4. Instructions perform both elementary and complex operations.

Reduced instruction set computers (RISCs)

A *RISC architecture* has the following properties:

1. All operations on data apply to data in registers and typically change the entire register (32 or 64 bits per register).

2. Memory accesses are restricted to load and store instructions, and data manipulation instructions are register-to-register.

3. Addressing modes are limited.

4. Instruction formats are all of the same length.

5. Instructions perform elementary operations.

Classes of Instructions:

- The goal of a RISC architecture is high throughput and fast execution. To achieve these goals, accesses to memory, which typically take longer than other elementary operations, are to be avoided, except for fetching instructions.

Most RISC architectures, have three classes of instructions:

1- *ALU instructions*—These instructions take either two registers or a register and a sign-extended immediate.

2- *Load and store instructions*

3- Branches and jumps

A Simple Implementation of a RISC Instruction Set

- To understand how a RISC instruction set can be implemented in a pipelined fashion, we need to understand how it is implemented *without* pipelining.
- Every instruction in this RISC subset can be implemented in at most 5 clock cycles.
- The 5 clock cycles are as follows:

  IF□ ID/ OF□EX□ MEM□WB

Cont.,

- IF instruction fetch
- ID / OF instruction decoding / operand fetch
- EX ALU execution
- MEM memory access if the current instruction is a Load, or Store (specific address is computed by Ex
- WB write the result in a register file.

	Clock number								
Instruction number	1	2	3	4	5	6	7	8	9
Instruction *i*	IF	ID	EX	MEM	WB				
Instruction *i* + 1		IF	ID	EX	MEM	WB			
Instruction *i* + 2			IF	ID	EX	MEM	WB		
Instruction *i* + 3				IF	ID	EX	MEM	WB	
Instruction *i* + 4					IF	ID	EX	MEM	WB

Sheet

- Text Book